Module 05

Introduction to DBMS/2

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Basic notions and terminology of database management systems
- Role of data models and languages
- Approaches to database design

Module Objectives

- To understand models of database management systems
- To familiarize with major components of a database engine
- To familiarize with database internals and architecture
- To understand the historical perspective

Module Outline

- Database Design

Module Outline

- Database Design
- OO Relational Model

Module Outline

- Database Design
- OO Relational Model
- XML

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine
    - Storage Management

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine
  - Storage Management
  - Query Processing

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine
  - Storage Management
  - Query Processing
  - Transaction Management

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine
  - Storage Management
  - Query Processing
  - Transaction Management
- Database Internals and Architecture

Module Outline

- Database Design
- OO Relational Model
- XML
- Database Engine
  - Storage Management
  - Query Processing
  - Transaction Management
- Database Internals and Architecture
- Database Users and Administrators

Database Design

Database Design

Defining the Structure

The process of designing the general structure of the database:

Database Design

Defining the Structure

The process of designing the general structure of the database:

- **Logical Design** – Deciding on the database schema. Database design requires that we find a good collection of relation schema
  - **Business decision**: What attributes should we record in the database?

Database Design

Defining the Structure

The process of designing the general structure of the database:

- **Logical Design** – Deciding on the database schema. Database design requires that we find a good collection of relation schema
  - **Business decision**: What attributes should we record in the database?
  - **Computer Science decision**: What relation schemas should we have and how should the attributes be distributed among the various relation schemas?

Database Design

Defining the Structure

The process of designing the general structure of the database:

- **Logical Design** – Deciding on the database schema. Database design requires that we find a good collection of relation schema
  - **Business decision**: What attributes should we record in the database?
  - **Computer Science decision**: What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **Physical Design** – Deciding on the physical layout of the database

Database Design (2)

Is there any problem with this relation?

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Database Design (3)

(a) The instructor table

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
...	...	...	...

(b) The department table

dept_name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

Design Approaches

Need to come up with a methodology to ensure that each relations in the database is good. Two ways of doing so:

Design Approaches

Need to come up with a methodology to ensure that each relations in the database is good. Two ways of doing so:

Methodologies

- **Entity Relationship Model** (Chapter 7)
  - Models an enterprise as a collection of entities and relationships

Design Approaches

Need to come up with a methodology to ensure that each relations in the database is good. Two ways of doing so:

Methodologies

- **Entity Relationship Model** (Chapter 7)
  - Models an enterprise as a collection of entities and relationships
  - Represented diagrammatically by an entity-relationship diagram

Design Approaches

Need to come up with a methodology to ensure that each relations in the database is good. Two ways of doing so:

Methodologies

- **Entity Relationship Model** (Chapter 7)
  - Models an enterprise as a collection of entities and relationships
  - Represented diagrammatically by an entity-relationship diagram
- **Normalization Theory** (Chapter 8)

Design Approaches

Need to come up with a methodology to ensure that each relations in the database is good. Two ways of doing so:

Methodologies

- **Entity Relationship Model** (Chapter 7)
  - Models an enterprise as a collection of entities and relationships
  - Represented diagrammatically by an entity-relationship diagram
- **Normalization Theory** (Chapter 8)
  - Formalize what designs are bad, and test for them

Object-Relational Data Models

Object-Relational Data Models

Limitations of Relational Model

Relational model explicitly limits storage to flat, atomic values.

Object-Relational Data Models

Limitations of Relational Model

Relational model explicitly limits storage to flat, atomic values.

Object Relational Extensions

- Extend the relational data model by including object orientation and constructs to deal with added data types
- Allow attributes of tuples to have complex types, including non-atomic values such as nested relations

Object-Relational Data Models

Limitations of Relational Model

Relational model explicitly limits storage to flat, atomic values.

Object Relational Extensions

- Extend the relational data model by including object orientation and constructs to deal with added data types
- Allow attributes of tuples to have complex types, including non-atomic values such as nested relations
- Preserve relational foundations, in particular the declarative access to data, while extending modeling power

Object-Relational Data Models

Limitations of Relational Model

Relational model explicitly limits storage to flat, atomic values.

Object Relational Extensions

- Extend the relational data model by including object orientation and constructs to deal with added data types
- Allow attributes of tuples to have complex types, including non-atomic values such as nested relations
- Preserve relational foundations, in particular the declarative access to data, while extending modeling power
- Provide upward compatibility with existing relational languages

XML: Extensible Markup Language

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange data, not just documents

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange data, not just documents
- XML has **become the basis** for all new generation data interchange formats

XML: Extensible Markup Language

- Defined by the WWW Consortium (W3C)
- Originally intended as a document markup language not a database language
- The ability to specify new tags, and to create nested tag structures made XML a great way to exchange data, not just documents
- XML has **become the basis** for all new generation data interchange formats
- A wide variety of tools is available for parsing, browsing and querying XML documents/data

Database Engine

Database Engine

Core Components

- Storage manager

Database Engine

Core Components

- Storage manager
- Query processing

Database Engine

Core Components

- Storage manager
- Query processing
- Transaction manager

Storage Management

The Storage Manager

A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

Storage Management

The Storage Manager

A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- The storage manager is responsible for:
  - Interaction with the OS file manager

Storage Management

The Storage Manager

A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- The storage manager is responsible for:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data

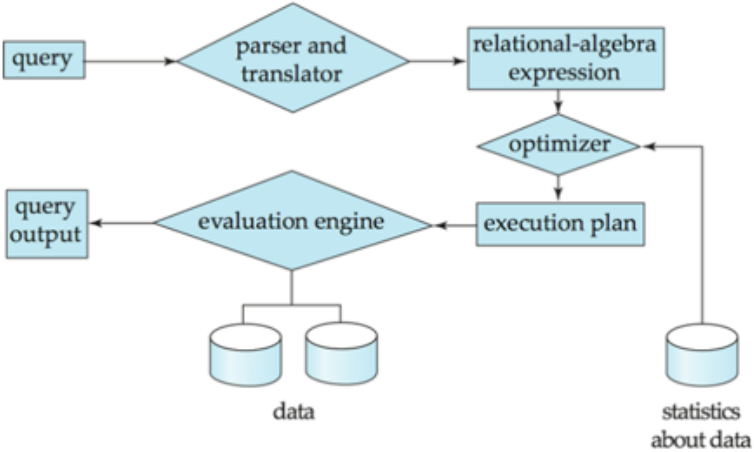
Storage Management

The Storage Manager

A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- The storage manager is responsible for:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data
- Issues: Storage access, File organization, Indexing and hashing

Query Processing



Query Processing (2)

- Alternative ways of evaluating a given query

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- **Cost difference** between a good and a bad way of evaluating a query can be enormous

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- **Cost difference** between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- **Cost difference** between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain

Query Processing (2)

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- **Cost difference** between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain
  - Need to estimate statistics for intermediate results to compute cost of complex expressions

Transaction Management

Challenges

What if the system fails? What if more than one user is concurrently updating the same data?

Transaction Management

Challenges

What if the system fails? What if more than one user is concurrently updating the same data?

- A **transaction** is a collection of operations that performs a single logical function in a database application.
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures and transaction failures.

Transaction Management

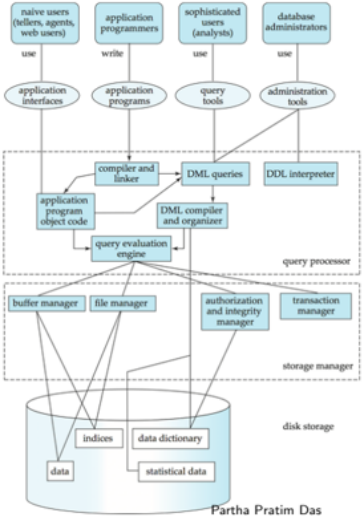
Challenges

What if the system fails? What if more than one user is concurrently updating the same data?

- A **transaction** is a collection of operations that performs a single logical function in a database application.
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures and transaction failures.
- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

Database System Internals

Database System Internals

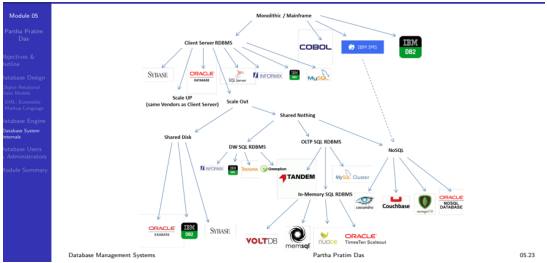


Database Architecture

The architecture of a database system is greatly influenced by the underlying computer system on which the database is running:

- Centralized
- Client-server
- Parallel (multi-processor)
- Distributed
- Cloud

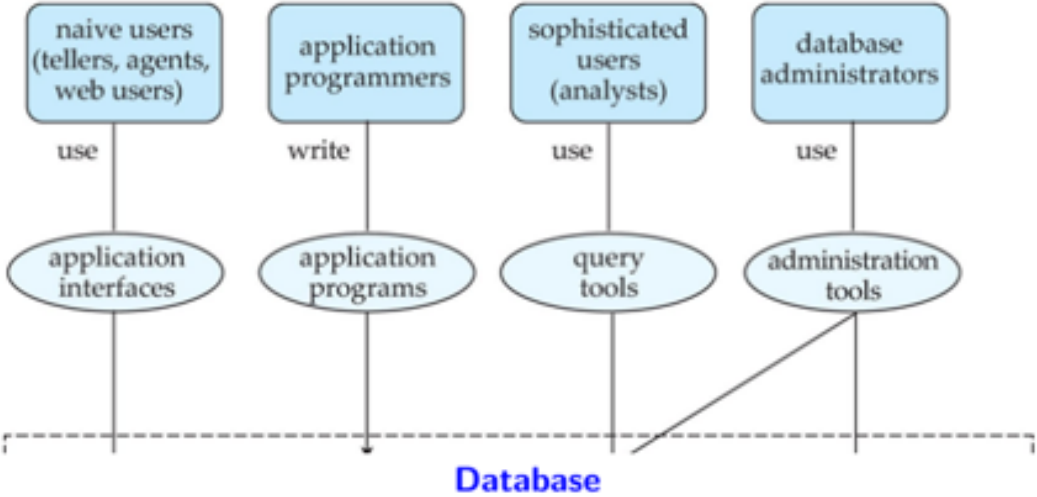
Database Architecture (2)



Database Users and Administrators

Database Users and Administrators

Database Users and Administrators (2)



Module Summary

- Introduced models of database management systems
- Familiarized with major components of a database engine
- Familiarized with database internals and architecture

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.